Architectural Analysis and Experimental Evaluation of Ceph Distributed Storage

Alessandro Claudio Damo
University of Padua
Email: alessandroclaudio.damo@studenti.unipd.it

***Abstract*—Ceph is a distributed storage system designed to provide scalability, reliability, and high performance by eliminating the traditional single points of failure typical of centralized storage systems. Its architecture relies on fully decentralized data placement, a clear separation between the data plane and the control plane, and distributed replication and recovery mechanisms operating at the Placement Group level. In this work, we analyze the architecture of Ceph, focusing on the role of RADOS, Object Storage Daemons, and the CRUSH algorithm, and we highlight how these design choices influence system performance. Through a series of controlled experiments conducted on a Ceph cluster, we empirically investigate the effects of the replication factor, the number of active OSDs, the granularity of Placement Groups, and object size on throughput and latency.**

I. Introduction

Ceph is an open-source distributed storage platform designed to guarantee scalability, reliability, and high performance by eliminating the traditional single points of failure typical of centralized storage systems. Originally designed to operate on commodity hardware, it enables the construction of large-scale storage systems that can grow dynamically according to demand without service interruption. Ceph also supports the *object storage model*, in which data are stored as independent objects within a flat namespace, without hierarchical structure, and are accessed not through POSIX calls (such as `open`, `read`, `write`), but via REST APIs (e.g., S3) and client libraries.

II. Core Components

A. RADOS

RADOS (Reliable Autonomic Distributed Object Store) is the core component of Ceph responsible for directly managing object storage within the cluster. It distributes data across the cluster, maintains replicas to ensure fault tolerance, and provides reliability, load balancing, and automatic data recovery in the event of node failures.

B. Object Storage Daemons (OSDs)

Object Storage Daemons (OSDs) represent the fundamental component of Ceph's storage layer and constitute the central element of the RADOS system.

In Ceph, data are managed as objects, and each OSD stores a subset of these objects on its local storage device, typically associated with a single storage device. From an implementation perspective, an OSD is a daemon running on a cluster host; a single host may run multiple OSDs, each associated with a different device.

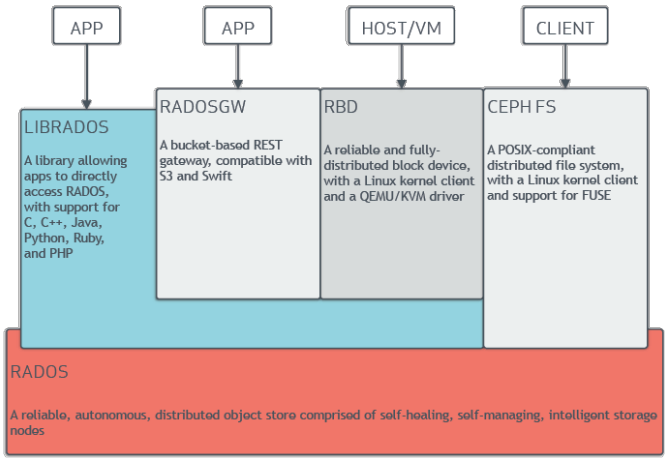

Fig. 1: The logical architecture of the Ceph stack. At the base lies RADOS (Reliable Autonomic Distributed Object Store), the fundamental storage layer responsible for managing object distribution, data replication, failure detection, and recovery operations within the cluster. Above RADOS, several storage access services are implemented.

A key characteristic of the Ceph architecture is that Object Storage Daemons are not passive storage endpoints, but active participants in the distributed protocol. Each OSD is responsible not only for storing data, but also for executing replica management, coordinating with peer OSDs, detecting failures, and participating in recovery operations.

C. Monitors (MONs)

Monitors (MONs) are daemons that run on multiple nodes of the cluster. They constitute the control plane of the Ceph system and are responsible for maintaining a consistent distributed view of the global state of the cluster. Unlike OSDs, MONs do not participate in the data path and do not handle data directly; their role is to maintain a consistent view of the system configuration and make it available to clients and OSDs.

The global cluster state is represented by the *cluster map*, a data structure that describes the cluster composi-

tion, OSD topology, mapping between Placement Groups and OSDs placement policies. Each change in the system state produces a new version of the map, identified by an *epoch* number, which is incrementally propagated to other components.

To guarantee consistency of the cluster map even in the presence of failures, Monitors operate as a replicated group and rely on a distributed consensus mechanism to agree on global state updates. This approach allows the system to tolerate the failure of one or more MONs, as long as quorum is maintained, preventing clients and OSDs from operating on divergent cluster views.

D. Clients

Clients represent the entry point to the distributed storage system and constitute an essential component of the Ceph architecture. They may run on remote nodes, communicating with the cluster over the network, or on the same hosts that run OSDs, without specific architectural constraints.

In the Ceph model, we will see that the client plays an active role both in computing data placement and in accessing the storage.

III. Objects, Placement Groups, and Pools

A. Objects

In the Ceph storage model, the object represents the smallest logical unit on which replication, recovery, and consistency mechanisms operate. At this level, Ceph does not reason in terms of files or logical blocks, but exclusively in terms of objects managed by RADOS.

Each object is composed of three fundamental elements:

- **Unique identifier (Object ID)**: used to uniquely identify the object
- **Metadata**: descriptive information associated with the object
- **Data**: the actual content of the object

Objects constitute the interface between higher-level services and the core storage layer of Ceph, enabling uniform access that is independent of the specific workload. From an architectural perspective, object management is completely distributed and their location is not tracked through centralized indexes, but is deterministically computed through the hashing and placement mechanisms described in the following sections.

B. Placement Groups

Placement Groups (PGs) represent an intermediate abstraction layer between objects and Object Storage Daemons (OSDs). A PG is not a physical container of data, but a logical entity that groups a set of objects and governs their behavior in terms of replication, recovery, and migration.

The introduction of Placement Groups is motivated by scalability requirements. In large-scale clusters, the number of objects can be extremely high, and managing placement and recovery at the level of individual objects would be impractical. By operating on a more limited number of PGs instead, Ceph can perform rebalancing and recovery operations in an efficient and controlled manner.

Each PG also has an associated state that reflects the consistency condition of the data it represents (e.g., *active*, *clean*, *degraded*, *recovering*), allowing the system to monitor the global state of the storage and automatically trigger recovery operations when necessary.

C. Pools

A pool is a logical container that defines the storage policies applied to a set of Placement Groups. All objects in Ceph are always associated with a pool, and the pool's characteristics determine system behavior in terms of reliability, data distribution, and resource utilization.

From an architectural perspective, the pool represents the point at which the main configuration decisions are specified, such as the replication factor, the use of traditional replication or erasure coding, the number of Placement Groups, and the CRUSH rules used for data placement. In this way, Ceph supports heterogeneous workloads within the same cluster without compromising the simplicity of the management model.

IV. CRUSH and Data Placement

CRUSH is the fundamental mechanism used by Ceph for decentralized data placement. It is designed to eliminate the use of centralized allocation tables and to enable system scalability.

The principle underlying CRUSH is that data locations are not looked up, but computed. Given an object identifier and a description of the cluster, data placement can be autonomously determined by any node that possesses the same configuration information.

A. Data placement

The first step of data placement in Ceph does not directly involve CRUSH. Each object is initially associated with a specific Placement Group (PG) through a deterministic hashing function applied to its identifier. This calculation step uniquely determines which PG the object belongs to.

As we already seen, the use of Placement Groups reduces system complexity: instead of mapping each individual object to physical devices, Ceph operates on a limited number of PGs, making rebalancing and recovery operations more manageable.

Once the Placement Group has been calculated, CRUSH comes into play. The algorithm uses the *CRUSH map*, which describes the organization of OSDs and data distribution rules, to compute the set of OSDs on which the PG and its replicas should be placed. This computation is fully deterministic: every node that possesses the same CRUSH map obtains the same result, without requiring centralized coordination or global allocation tables.

Since data placement is entirely computable from the CRUSH map, which is distributed to clients and OSDs by distributed monitors, there is no central component responsible for tracking data locations. Clients and OSDs can therefore autonomously determine placement, eliminating bottlenecks and single points of failure in the data path.

B. Changes in cluster composition

When the cluster composition changes, for example due to the addition or removal of an Object Storage Daemon, the Monitors update the global system state by producing a new version of the cluster map and the CRUSH map. CRUSH does not detect or manage these events directly, but instead uses the updated map to deterministically recompute data placement.

The recomputation of Placement Group locations follows a mapping stability policy: in response to a configuration change, only a limited fraction of PGs is remapped, while the majority of the data remains in its original location. This behavior drastically reduces the amount of data involved in rebalancing operations and allows the system to adapt dynamically to changes without compromising performance or scalability.

V. Data Plane and Control Plane Separation

A fundamental architectural principle underlying the design of Ceph is the clear separation between the data plane and the control plane.

The *data plane* includes all I/O operations on user data and involves exclusively the clients and the Object Storage Daemons (OSDs). As discussed previously, in this plane clients autonomously compute object placement using CRUSH and communicate directly with the responsible OSDs, without traversing centralized components. Read and write operations therefore occur in a fully distributed manner, allowing system throughput to scale with the number of storage devices.

The *control plane* is instead handled by the Monitors (MONs) and is responsible for managing the global state of the cluster. The MONs maintain and distribute the cluster maps, which describe system composition, OSD topology, and placement rules. State updates are coordinated through a consensus mechanism, ensuring that all nodes operate on a consistent view of the system.

This two separation and distributed coordination keeps the data path free of dependencies on central components and bottlenecks. As a result, Ceph can provide horizontal scalability, high availability, and fault tolerance even in large-scale clusters.

VI. Experimental Methodology

The experiments presented in this work were conducted on a Ceph cluster configured in a controlled environment, with the goal of analyzing system behavior and isolating the effects of the main architectural choices.

The cluster was deployed on a single physical host equipped with one storage device, but we configured multiple loop-backed devices on the same physical storage medium, in order to simulate several storage device. Each loop device was then initialized as a distinct Object Storage Daemon (OSD), the Monitor (MON) daemons and the client were also executed on the same host.

It is important to note that, due to the presence of a single physical storage device, this setup represents a significant limitation. This effect limits the degree of effective hardware parallelism that can be achieved and prevents the results from reflecting the performance of a real distributed deployment. This effect introduces a distortion in the absolute parallelism values observed, and the results obtained do not represent the absolute performance of a real distributed cluster. Nevertheless, with these limitations in mind, the setup still allows for a clear and reproducible analysis of performance trends and the architectural behavior of Ceph.

VII. Replication

Replication is the mechanism through which Ceph guarantees data reliability and availability within the cluster. The system maintains multiple copies of the same data, distributed across different Object Storage Daemons (OSDs), in order to tolerate hardware or software failures without data loss.

In Ceph, replication does not operate at the level of individual objects, but at the level of Placement Groups (PGs). Each PG is associated with a set of OSDs, called the *acting set*, within which one OSD assumes the role of *primary* and coordinates write operations toward the replica OSDs, ensuring strong consistency according to a primary-based model.

The number of replicas is defined by the pool parameter `size` and determines how many copies of each PG must be maintained. Replicas play a fundamental role in fault tolerance, as they allow the system to preserve data availability when the system faces disk or node failures. In the event of a fault, Ceph uses the remaining valid replicas to reconstruct missing copies on other OSDs, thereby restoring the configured level of redundancy.

A. Replication Experiment

To evaluate the performance impact of replication, we used Ceph's built-in rados bench utility [4] to measure latency and throughput as the replication factor increased.

As shown in Fig. 2, increasing the replication factor resulted in a significant reduction in the observed throughput. In particular, under the same workload and cluster configuration, increasing the number of replicas led to a higher number of write operations required to guarantee consistency, consequently reducing the effectively available bandwidth.

In production environments, the cost of replication is partially masked by parallelism across physically separate

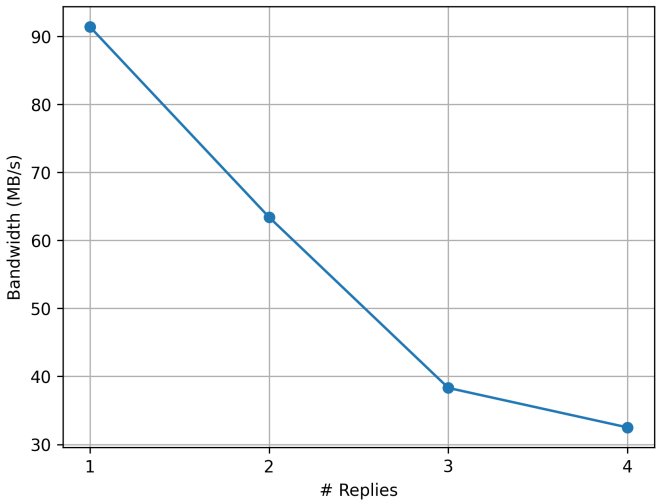

Fig. 2: Bandwidth as a function of the number of replicas. As the replication factor increases, the available bandwidth decreases due to the additional write operations required to guarantee data consistency.

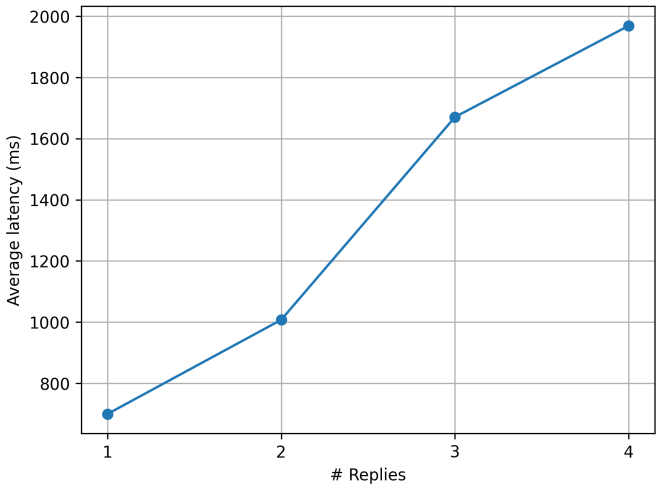

Fig. 3: Average latency as a function of the number of replicas. Increasing the replication factor leads to higher latency, as each write operation must be acknowledged by a larger number of replicas.

OSDs. In our experimental setup, however, replicas compete for the same hardware resources, and increasing the replication factor results in a more pronounced reduction in throughput. Regarding latency, our experiments show an increase in the average write latency as the replication factor grows (Fig. 3). In the considered configuration, the primary OSD must wait for acknowledgments from all replicas before completing the operation, leading to an increase in the average response time. Therefore, these results are consistent with those obtained for the same experiment in the paper [1].

VIII. Parallelism and OSD Participation

# Available OSDs	# Active OSDs
1	**1**
2	**2**
3	**3**
4	**4**
5	**5**

TABLE I: Number of OSDs participating in write operations as the number of available OSDs varies

A. Parallelism Experiment

With the replication factor fixed at two and the number of Placement Groups set to 16, the same write benchmark previously used was executed using the `rados bench` utility within Ceph [4], which generates synthetic write operations directly at the RADOS level. During the experiment, the number of Object Storage Daemons (OSDs) was progressively increased from one to five, while keeping all other test parameters constant. For each configuration, we observed how many OSDs actually participated in the workload, that is, how many performed at least one write operation.

The results show that, as reported in Table I, increasing the number of available OSDs leads to the participation of all OSDs in write operations, even though the replication factor is set to two, a value that would imply the simultaneous involvement of only two OSDs per write operation.

This behavior can be explained by considering that, although each individual write operation involves a number of OSDs equal to the replication factor (`size`), the execution of many write operations distributed across different Placement Groups leads, over time, to the participation of all available OSDs in the overall workload.

In general, Ceph tends to distribute operations as uniformly as possible, involving the largest possible number of available OSDs in order to maximize parallelism and balance the load.

B. What Actually Happens

In Ceph, write operations are directed to a Placement Group (PG), and PGs are distributed among Object Storage Daemons (OSDs) according to the CRUSH algorithm. As discussed previously, CRUSH uses a deterministic hashing function that produces a pseudo-random distribution of PGs across OSDs, avoiding hotspots and load concentration while promoting a distribution of the workload that is as uniform as possible.

Each OSD therefore represents a potential unit of parallelism. In Ceph, every OSD is an active and independent process, equipped with its own request queue, which utilizes the hardware resources of the host on which it runs, in particular CPU, memory, and the storage device where I/O operations are performed. Operations directed to different OSDs can therefore be processed concurrently,

although the effective level of parallelism ultimately depends on the hardware resources available on the host.

In CRUSH, each object is assigned to a PG, which is then mapped to a set of OSDs in an approximately uniform manner. During the writing of a sequence of objects, these are therefore assigned to different PGs distributed across the OSDs in the cluster. When an OSD becomes involved in a write operation, it becomes an active element of parallelism, as its request queue and the computational resources of the host are engaged in processing the operation.

This mechanism leads to an increase in the aggregate parallelism of the system: during a sequence of object write operations, progressively more OSDs become involved, allowing a larger portion of the cluster resources and available bandwidth to be utilized.

In this sense, Ceph scales horizontally not by making a single device faster, but by using many devices concurrently.

C. Reference to the literature

This behavior is consistent with what is reported in the paper *Ceph: A Scalable, High-Performance Distributed File System*, which shows that the aggregate cluster throughput grows almost linearly with the number of OSDs, provided that the network is not saturated and that the number of Placement Groups is sufficient to distribute the load.

The Ceph architecture, based on CRUSH and a fully distributed data path, therefore allows the overall throughput to scale approximately proportionally with the addition of new OSDs, until other hardware or network bottlenecks emerge.

Naturally, this effect has a limit: once components such as the network, OSD CPUs, clients, or an insufficient number of PGs become limiting factors, increasing the number of OSDs no longer results in a proportional performance improvement. However, before reaching these limits, the progressive activation of an increasing number of OSDs over time represents the key mechanism through which Ceph increases aggregate throughput.

IX. Placement Group Balancing

After observing that, over time, write operations in Ceph tend to involve all Object Storage Daemons (OSDs) in the cluster, it is necessary to analyze how the system guarantees load and capacity balancing across different storage nodes.

Balancing is a fundamental mechanism for scalability and efficient resource utilization in distributed storage systems. Capacity and workload must be evenly distributed across nodes, avoiding situations in which some devices become bottlenecks while others remain underutilized. Ceph addresses this problem by adopting an automatic and fully distributed approach, without introducing centralized components dedicated to balancing.

In Ceph, balancing does not occur at the level of individual objects, but at the level of Placement Groups (PGs). PGs represent the logical unit for data distribution and replication and are mapped onto OSDs through the CRUSH algorithm. This abstraction level makes balancing and rebalancing manageable even in large-scale clusters.

The distribution of PGs across OSDs, determined by CRUSH based on a pseudo-random hashing function and the weights associated with devices, ensures that each OSD hosts a fraction of the Placement Groups. Over time, this leads to a statistically uniform distribution of read and write operations. However, this uniformity depends on the granularity of data subdivision, namely the total number of Placement Groups.

With a small number of PGs, the distribution is coarse-grained and some OSDs may receive a significantly larger share of data or workload than others. Conversely, increasing the number of Placement Groups results in a finer-grained distribution that tends to become more uniform, improving load balancing and enabling more efficient utilization of cluster resources.

A. Balancing Experiment

After observing that, over time, write operations in Ceph tend to involve all Object Storage Daemons (OSDs) in the cluster, this experiment aims to analyze how the quality of data balancing depends on the total number of Placement Groups (PGs), while keeping the workload and replication factor constant.

The experiment was conducted by creating two replicated pools, both configured with a constant replication factor (`size = 2`) and `minsize = 1`, differing exclusively in the number of Placement Groups. In the first case, a limited number of PGs was used (LOW PG, 32 PGs), while in the second case a higher number was adopted (HIGH PG, 128 PGs). The pools were used exclusively for benchmarking and were cleaned between tests in order to eliminate interference from pre-existing data.

The workload applied to the two pools was identical and was generated using the `rados bench` command, executing write operations for 120 seconds with 16 concurrent threads and a block size of 4 MB. Under these conditions, the RAW USE per OSD represents a reliable indicator of the amount of data assigned to each OSD by the placement mechanism and can therefore be used to evaluate the degree of balancing.

In the case of the pool with 32 Placement Groups, the distribution of RAW USE across OSDs shows significant variability. Utilization percentages range from 21.7% to 29.5% of the total, with a standard deviation of 3.49. The ratio between the minimum and maximum values relative to the ideal average (25%) highlights a non-negligible deviation, indicating that some OSDs receive a substantially larger or smaller share of data than others.

This behavior is consistent with a coarse-grained distribution, in which the limited number of PGs reduces place-

TABLE II: Distribution of raw space usage and effective data allocation across OSDs with a limited number of Placement Groups (16 PGs). The table shows the resulting imbalance in space utilization due to the small number of PGs, which prevents uniform data distribution among OSDs.

OSD	DATA (GiB)	% RAW USE
osd.1	3.8	29.5%
osd.2	2.8	21.7%
osd.3	3.1	24.0%
osd.4	3.2	24.8%
Total	**12.9**	**100%**

MIN/MAX VAR: 0.88 / 1.17 STDDEV: 3.49

TABLE III: Distribution of effective data allocation and raw space usage across OSDs with a high number of Placement Groups. The table shows a more uniform space utilization compared to the low-PG configuration, as the larger number of PGs enables finer-grained and more balanced data distribution among OSDs.

OSD	DATA (GiB)	% RAW USE
osd.1	3.0	24.4%
osd.2	3.2	26.0%
osd.3	3.4	27.6%
osd.4	2.7	22.0%
Total	**12.3**	**100%**

MIN/MAX VAR: 0.89 / 1.11 STDDEV: 2.66

ment granularity and makes imbalances in data allocation more likely.

By increasing the number of Placement Groups to 128, data distribution becomes more uniform. In this scenario, RAW USE percentages per OSD range between 22.0% and 27.6%, with a reduced standard deviation of 2.66. The lower dispersion of values indicates that each OSD receives a share of data closer to the ideal average.

Increasing the number of PGs therefore improves distribution granularity, allowing CRUSH to partition data more finely and reducing imbalances among OSDs.

The comparison between the two scenarios clearly shows that simply involving all OSDs over time is not sufficient to guarantee uniform data balancing. The quality of balancing depends critically on the total number of Placement Groups, which determines the granularity with which data are subdivided and distributed across OSDs.

With a limited number of PGs, placement is coarser and leads to higher variability in the amount of data assigned to each OSD. Conversely, a higher number of PGs enables a more uniform distribution, improving balancing and creating the conditions for more efficient resource utilization and more effective I/O parallelism.

X. Recovery and Rebalancing

When the cluster composition changes—for example due to the addition of new disks or new nodes—Ceph automatically recomputes the mapping between Placement Groups (PGs) and Object Storage Daemons (OSDs). The CRUSH algorithm determines a new distribution that takes into account the available resources and the placement rules defined, such as failure domains. In this process, only a fraction of the PGs is remapped to new OSDs, limiting the amount of data that must be transferred and reducing the impact on system performance.

Similarly, following the removal of an OSD from the cluster, recovery of the affected Placement Groups relies exclusively on replicas that are still present on the remaining OSDs. Ceph is able to reconstruct data only if at least one valid replica of each PG exists within the cluster: in this case, healthy replicas are used as a source to automatically regenerate missing copies on the OSDs selected by the placement algorithm, restoring the configured level of redundancy. If no valid replicas are available, recovery is not possible and the data are permanently lost.

A. Recovery Experiment

We empirically analyzed Ceph's data recovery mechanism following the removal of an Object Storage Daemon (OSD) from the cluster, observing how Placement Groups (PGs) are reallocated and how the system restores the configured replication level.

The experiment considers a cluster composed of a single host with five OSDs (`osd.0`–`osd.4`), a pool named `testpool`, a replication factor of two (`size = 2`), and a limited number of Placement Groups (`pgnum = 8`) to facilitate observation of system behavior.

In the initial state, the cluster is stable and all Placement Groups are in the `active+clean` state. For each PG, the `UP` set coincides with the `ACTING` set, indicating that all expected replicas are present and correctly placed. It can also be observed that `osd.3` participates in a significant number of PGs, making its removal a suitable event for studying recovery behavior.

A fault event was then triggered by removing an OSD from the cluster using the command `ceph osd out osd.3`.

Following this command, `osd.3` is excluded from the CRUSH map. All Placement Groups that previously included it in their acting set must therefore restore the configured replication factor.

Immediately after the OSD removal, the affected Placement Groups enter transient states such as `active+degraded+recovering`. At this stage, it is essential to distinguish between the `UP` and `ACTING` sets.

The `UP` set represents the new target calculated by CRUSH and no longer includes `osd.3`, whereas the `ACTING` set may temporarily still include the removed OSD as long as a valid replica exists from which data can be read. During recovery, Ceph reads data from the remaining healthy replicas and reconstructs the missing copies on the OSDs indicated in the `UP` set. The removed OSD is no longer used as a data source.

It is important to emphasize that replicas are not "moved," but regenerated from valid copies already present in the cluster.

At the end of the recovery process, the cluster reaches a new stable state. All Placement Groups return to the `active+clean` state, and for each PG the `UP` and `ACTING` sets coincide again. The removed OSD (`osd.3`) no longer appears in any Placement Group, and the replication factor is fully restored.

Only the Placement Groups that involved the removed OSD experience a change in their acting set, while all other PGs remain unchanged. This demonstrates how Ceph limits the impact of recovery to only the data actually affected by the failure.

B. Implications for Balancing and Fault Tolerance

The experiment highlights that, following the removal of an OSD, Placement Groups are not transferred as physical entities, but instead modify their acting sets. Replicas that were previously located on the removed OSD are regenerated on the remaining OSDs selected by CRUSH, while replicas that are already valid remain in place. As a result, only a fraction of PGs is reallocated, and the cluster progressively converges toward a new equilibrium.

The recovery process occurs gradually and transparently. During replica reconstruction, Ceph keeps existing copies active until the new replicas have been fully synchronized, ensuring service continuity and data availability. Once reconstruction is complete, obsolete replicas are removed and the cluster reaches a stable state.

This behavior highlights the fully distributed nature of Ceph's fault-tolerance mechanisms. There is no central component coordinating data movement: clients and OSDs, using the maps provided by the Monitors, can locally determine PG placement and interact directly with the correct OSDs. In this sense, recovery and rebalancing represent two closely related aspects of the same architecture, contributing significantly to system scalability and resilience.

XI. Small Object Problem

The issue of small files is well known in file systems and distributed storage systems and is widely documented in the literature [3]. In general, when a system must manage a large number of small entities, the fixed cost associated with each operation tends to dominate over the transfer of useful data, resulting in a degradation of overall performance.

This phenomenon is particularly relevant in Ceph, as an object storage system. In Ceph, each piece of data is stored as an object, and each object introduces a per-operation overhead that is largely independent of the size of the data it contains. This overhead includes placement computation (Placement Group and OSD acting set), object metadata management, and operation-related costs such as replication, coordination, and journaling.

As a result, for a fixed total amount of transferred data, splitting the workload into smaller objects increases the number of operations and significantly raises the relative weight of overhead compared to useful data.

In this scenario, the system can progressively enter a condition in which the fixed overhead per operation almost completely saturates the total throughput, making the effective throughput extremely small.

The effective throughput decreases not because Ceph has fewer available resources, but because an increasing portion of the overall work is devoted to operation management (metadata handling, replication, and coordination) rather than to the actual transfer of data.

From an experimental perspective, reducing the object size parameter (block size) for each I/O operation causes the benchmark to generate a larger number of write operations, each carrying a smaller amount of useful data.

This behavior can be interpreted by considering a simplified model in which the total execution time is given by:

$$T_{\text{tot}} \approx N \cdot (T_{\text{fixed}} + T_{\text{variable}}) \tag{1}$$

where N is the number of operations, T_{fixed} represents the per-object overhead (metadata management, replication, journaling, and coordination), and T_{variable} is the time proportional to the amount of data transferred.

For the same total volume of data, reducing the object size results in an increase in N.

Consequently, the term $N \cdot T_{\text{fixed}}$ tends to dominate over the contribution proportional to the useful data, leading the system to reduce the effective throughput.

A. Small Objects Experiment

To evaluate this phenomenon, we conducted a series of benchmarks on a Ceph cluster composed of four OSDs, using a single dedicated RADOS pool (`benchpool`). The replication factor and the number of concurrent threads were kept constant throughout the experiments, with the latter fixed at 16. Each test was executed using the same `rados bench` with different I/O sizes (`-b`). After each test, the pool was completely cleaned to avoid interference from residual data and to ensure result comparability.

The results shown in Fig.4 exhibit a monotonic increase in throughput as the size of write operations increases. For very small sizes (16–32 KB), the observed bandwidth is extremely limited, on the order of a few MB/s. As object size increases, throughput grows significantly, reaching values above 140 MB/s with 4 MB writes.

This behavior clearly indicates that, the system may progressively enter a condition in which performance becomes dominated by fixed per-operation overhead rather than useful data transfer. As object size increases, this overhead is amortized over a larger amount of data, and the system progressively becomes bandwidth-bound, making more effective use of available I/O resources.

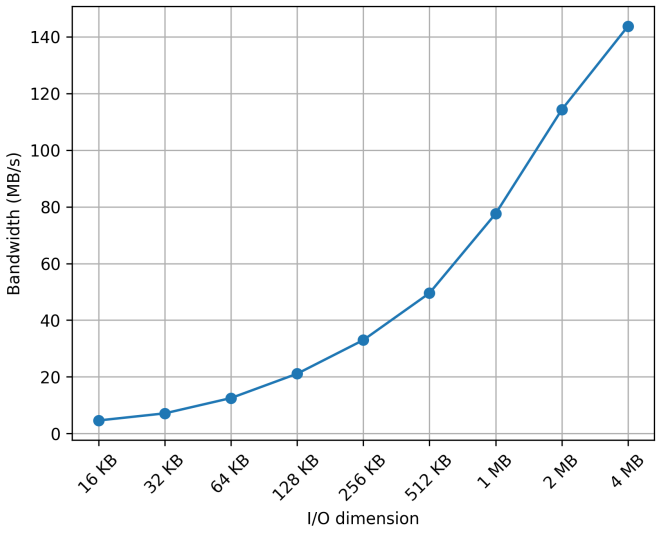

Fig. 4: Bandwidth as a function of I/O size. Throughput increases monotonically with object size, indicating a transition from an overhead-bound regime for small I/O sizes to a bandwidth-bound regime for larger objects.

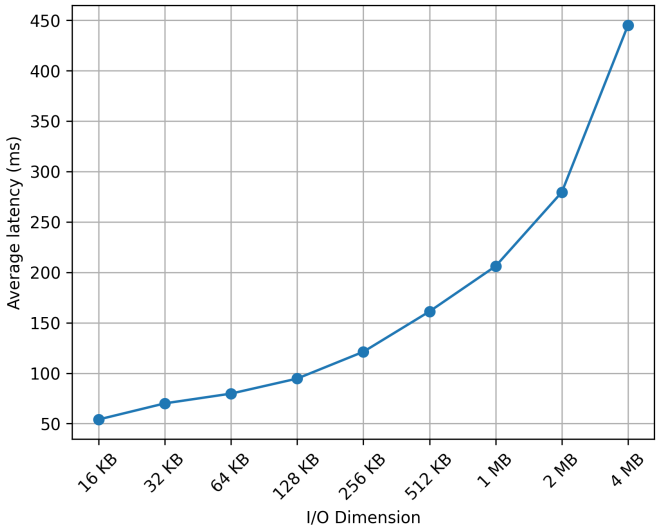

Fig. 5: Average latency as a function of I/O size. Increasing object size leads to higher latency, consistent with the larger volume of data transferred per operation.

The standard deviation of throughput also increases with object size, reflecting greater performance variability as the system approaches the physical limits imposed by storage devices and the network.

Analysis of average latency shows a complementary trend (Fig.5). As write size increases, per-operation latency grows significantly, from approximately 54 ms for 16 KB objects to over 445 ms for 4 MB objects. This increase is consistent with the fact that each operation transfers a larger amount of data and therefore remains in execution for a longer time.

It is important to note that the increase in latency does not represent a degradation in system behavior, but rather a direct consequence of larger operation sizes. In other words, the system trades per-operation latency for aggregate throughput: fewer operations, each more expensive in terms of time, but overall more efficient in data transfer.

Overall, the experiment clearly highlights the trade-off between object size, throughput, and latency. With small objects, Ceph is constrained by fixed per-operation overhead, which represents a significant fraction of the total work and achieves very low throughput; with larger objects, overhead is amortized, throughput increases significantly at the cost of higher per-operation latency.

These results experimentally confirm the small object problem discussed earlier and show how, in an object storage system such as Ceph, object size represents a crucial parameter for achieving good performance.

The experiment demonstrates that, in Ceph RADOS, object size is a critical performance parameter. Larger objects allow the cluster to maximize throughput, whereas small objects lead to excessive fragmentation that makes per-operation overhead dominant. This result provides a direct experimental explanation of the small object problem and justifies the use of aggregation or striping techniques for data-intensive workloads.

XII. Conclusions

This work analyzed the architecture of Ceph, highlighting how its main design choices—decentralized data placement, separation between the data plane and the control plane, and replication at the Placement Group level—contribute to scalability, reliability, and fault tolerance.

Through a series of targeted experiments, it was possible to empirically observe system behavior under different operating scenarios. In particular, increasing the replication factor was shown to incur a measurable performance cost in terms of throughput and latency, consistent with the synchronous, primary-based replication model adopted by RADOS. It was also shown that parallelism in Ceph does not arise from increased parallelization of a single I/O operation, but rather emerges over time from the pseudo-random distribution of Placement Groups across OSDs, enabling the system to effectively exploit independent resources.

Analysis of balancing and recovery demonstrated that, following changes in cluster composition, only a limited fraction of Placement Groups is reallocated, reducing the impact of rebalancing operations and enabling rapid convergence toward a new stable state. The recovery experiment further confirmed that Ceph reconstructs missing replicas from existing valid copies, without unnecessary data movement and while maintaining service availability throughout the process.

Finally, the study of the *small object problem* showed that, for small object sizes, the system enters an overhead-bound regime in which fixed per-operation costs dominate useful data transfer, limiting overall performance. This result emphasizes the importance of appropriate configuration choices and object sizing according to workload characteristics.

Overall, the experimental results confirm the scalability and resilience properties described in the literature on Ceph and demonstrate how the system architecture effectively adapts to different workloads and configurations, maintaining predictable and robust behavior.

References

[1] D. Thain, *Ceph Distributed Storage*, University of Notre Dame.

[2] S. A. Weil et al., "Ceph: A Scalable, High-Performance Distributed File System," OSDI.

[3] Metadata-heavy Workloads in Distributed File Systems, arXiv:2312.17524.

[4] Ceph Documentation, "rados — Ceph object storage utility (including rados bench)," Ceph Foundation. [Online]. Available: https://docs.ceph.com/en/latest/man/8/rados/

[5] Ceph Documentation, "Ceph Architecture," Ceph Foundation. [Online]. Available: https://docs.ceph.com/en/reef/architecture/